



Velociraptor

Learn Velociraptor, an advanced open-source endpoint monitoring, digital forensic and cyber response platform.

Task 1 – Introduction

Velociraptor

In this room, we will explore Rapid7's newly acquired tool known as [Velociraptor](#).

Per the official Velociraptor [documentation](#), "*Velociraptor is a unique, advanced open-source endpoint monitoring, digital forensic and cyber response platform. It was developed by Digital Forensic and Incident Response (DFIR) professionals who needed a powerful and efficient way to hunt for specific artifacts and monitor activities across fleets of endpoints. Velociraptor provides you with the ability to more effectively respond to a wide range of digital forensic and cyber incident response investigations and data breaches*".

This tool was created by Mike Cohen, a former Google employee who worked on tools such as [GRR](#) (GRR Rapid Response) and [Rekall](#) (Rekall Memory Forensic Framework). Mike joined Rapid7's Detection and Response Team and continues to work on improving Velociraptor. At the date of this entry, the latest release for Velociraptor is [0.6.3](#).

Learning Objectives

- Learn what is Velociraptor
- Learn how to interact with agents and create collections
- Learn how to interact with the virtual filesystem
- Learn what is VQL and how to create basic queries
- Use Velociraptor to perform a basic hunt

Prerequisites

- [Windows Forensics 1](#)
- [Windows Forensics 2](#)
- [KAPE](#)

Task 2 – Deployment

Deploying Velociraptor

Velociraptor is unique because the Velociraptor executable can act as a **server** or a **client** and it can run on **Windows**, Linux, and **MacOS**. Velociraptor is also compatible with cloud file systems, such as **Amazon EFS** and **Google Filestore**.

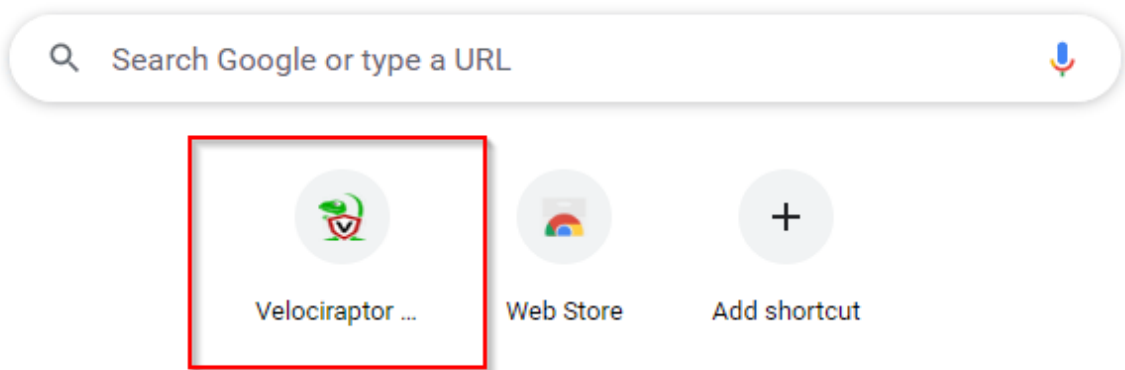
Velociraptor can be deployed across thousands, even tens of thousands, client endpoints and runs surprisingly well for an open-source product.

In this task, we will **NOT** go into detail about how to deploy Velociraptor as a server and agent architecture in an environment. Rather, in the attached lab machine, you will run the commands to start the first Velociraptor executable as a server and execute a second Velociraptor executable to run as an agent. This is possible thanks to WSL ([Windows Subsystem for Linux](#)). This will simulate Velociraptor running as a server in Linux (Ubuntu) and as a client running Windows. WSL (Windows Subsystem for Linux) allows us to run a Linux environment in a Windows machine without the need for a lab machine.

```
Start the Velociraptor Server (Ubuntu Terminal)

tryhackme@thm-velociraptor:~/velociraptor$ ./velociraptor-v0.5.8-linux-amd64 --config server.config.yaml frontend -v
[INFO] 2024-05-26T11:44:45Z
[INFO] 2024-05-26T11:44:45Z | | / / _ / _ _ _ _ _ _ _ _ / / _ _ _ _
[INFO] 2024-05-26T11:44:45Z | | / / _ V / _ V _ / / _ / _ _ ' / _ V _ / _ V _ /
[INFO] 2024-05-26T11:44:45Z | | / / _ / / / / / / / / / / / / / / / / / / / /
[INFO] 2024-05-26T11:44:45Z | _ \ / _ \ _ \ _ \ / / / \ , / . _ \ _ \ _ \ / /
[INFO] 2024-05-26T11:44:45Z / /
[INFO] 2024-05-26T11:44:45Z Digging deeper! https://www.velocidex.com
[INFO] 2024-05-26T11:44:45Z This is Velociraptor 0.5.8 built on 2021-04-11T22:09:54Z (e468f54c)
[INFO] 2024-05-26T11:44:45Z Loading config from file server.config.yaml
[INFO] 2024-05-26T11:44:45Z Starting Frontend. {"build_time":"2021-04-11T22:09:54Z","commit":"e468f54c","version":"0.5.8"}
[INFO] 2024-05-26T11:44:45Z Error increasing limit invalid argument. This might work better as root.
[INFO] 2024-05-26T11:44:45Z Starting Journal service.
[INFO] 2024-05-26T11:44:45Z Starting the notification service.
[INFO] 2024-05-26T11:44:45Z Starting Inventory Service
[INFO] 2024-05-26T11:44:45Z Loaded 250 built in artifacts in 88.9704ms
[INFO] 2024-05-26T11:44:45Z Starting Label service.
[INFO] 2024-05-26T11:44:45Z Selected frontend configuration localhost:8000
[INFO] 2024-05-26T11:44:45Z Starting Client Monitoring Service
[INFO] 2024-05-26T11:44:45Z Reloading client monitoring tables from datastore
[INFO] 2024-05-26T11:44:45Z Starting the hunt manager service.
[INFO] 2024-05-26T11:44:45Z Starting Hunt Dispatcher Service.
[INFO] 2024-05-26T11:44:45Z Starting Enrollment service.
[INFO] 2024-05-26T11:44:45Z server_monitoring: Starting Server Monitoring Service
[INFO] 2024-05-26T11:44:45Z Closing Server Monitoring Event table
[INFO] 2024-05-26T11:44:45Z server_monitoring: Updating monitoring table
[INFO] 2024-05-26T11:44:45Z server_monitoring: Collecting Server.Monitor.Health/Prometheus
[INFO] 2024-05-26T11:44:45Z Starting VFS writing service.
[INFO] 2024-05-26T11:44:45Z Starting Server Artifact Runner Service
[INFO] 2024-05-26T11:44:45Z Starting gRPC API server on 127.0.0.1:8001
[INFO] 2024-05-26T11:44:45Z Launched Prometheus monitoring server on 127.0.0.1:8003
[INFO] 2024-05-26T11:44:45Z GUI is ready to handle TLS requests on https://127.0.0.1:8889/
[INFO] 2024-05-26T11:44:45Z server_monitoring: Finished collecting Server.Monitor.Health/Prometheus
[INFO] 2024-05-26T11:44:45Z Query Stats: {"RowsScanned":1,"PluginsCalled":1,"FunctionsCalled":0,"ProtocolSearch":0,"ScopeCopy":5}
[INFO] 2024-05-26T11:44:45Z Frontend is ready to handle client TLS requests at https://localhost:8000/
[INFO] 2024-05-26T11:44:46Z Compiled all artifacts.
```

It's worth noting that the version of Velociraptor running in the attached lab machine is **0.5.8**. Now launch Google Chrome and click the Velociraptor shortcut.

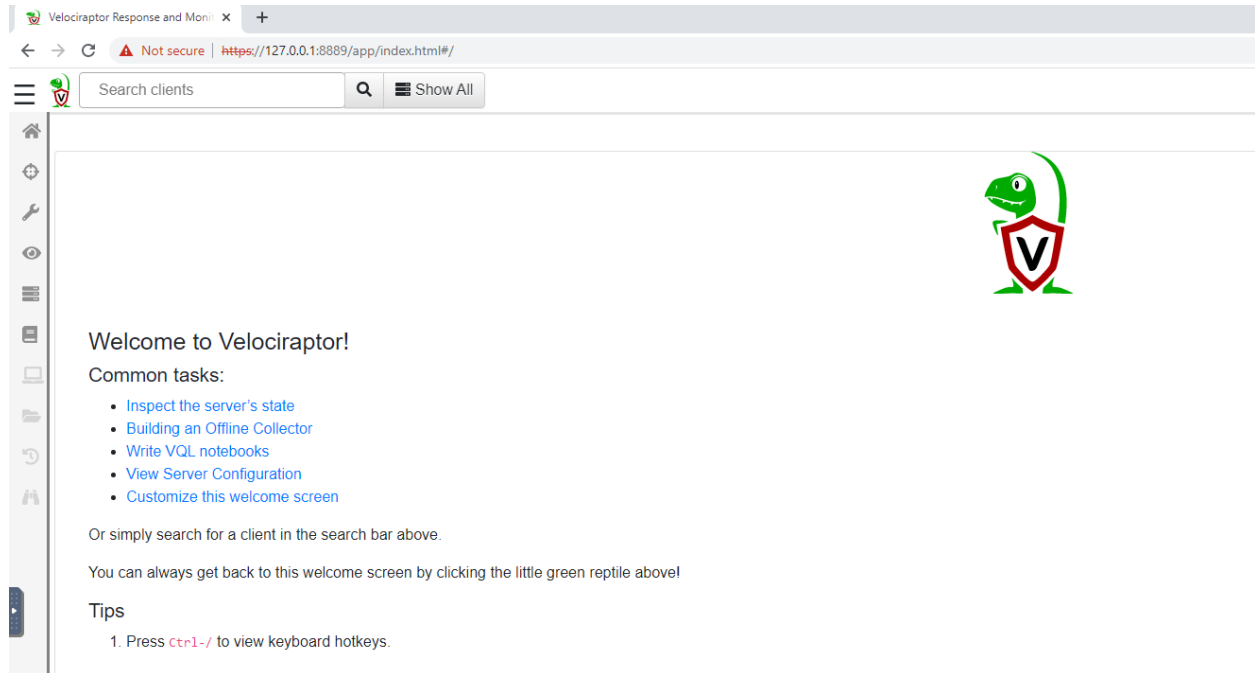


Chrome is likely to show you "Your Connection is not private errors", this is expected and you can proceed to 127.0.01 via the advanced option.

The credentials for the Velociraptor server are:

- Username: *****
- Password: *****

If all goes well, you should see the Velociraptor [Welcome screen](#).



If you wish to interact and deploy Velociraptor locally in your lab, then [Instant Velociraptor](#) is for you. Instant Velociraptor is a fully functional Velociraptor system that is deployed only to your local machine.

Refer to the official [documentation](#) for more information on deploying Velociraptor as a server/client infrastructure or as Instant Velociraptor.

Answer the questions below

Using the [documentation](#), how would you launch an Instant Velociraptor on Windows?

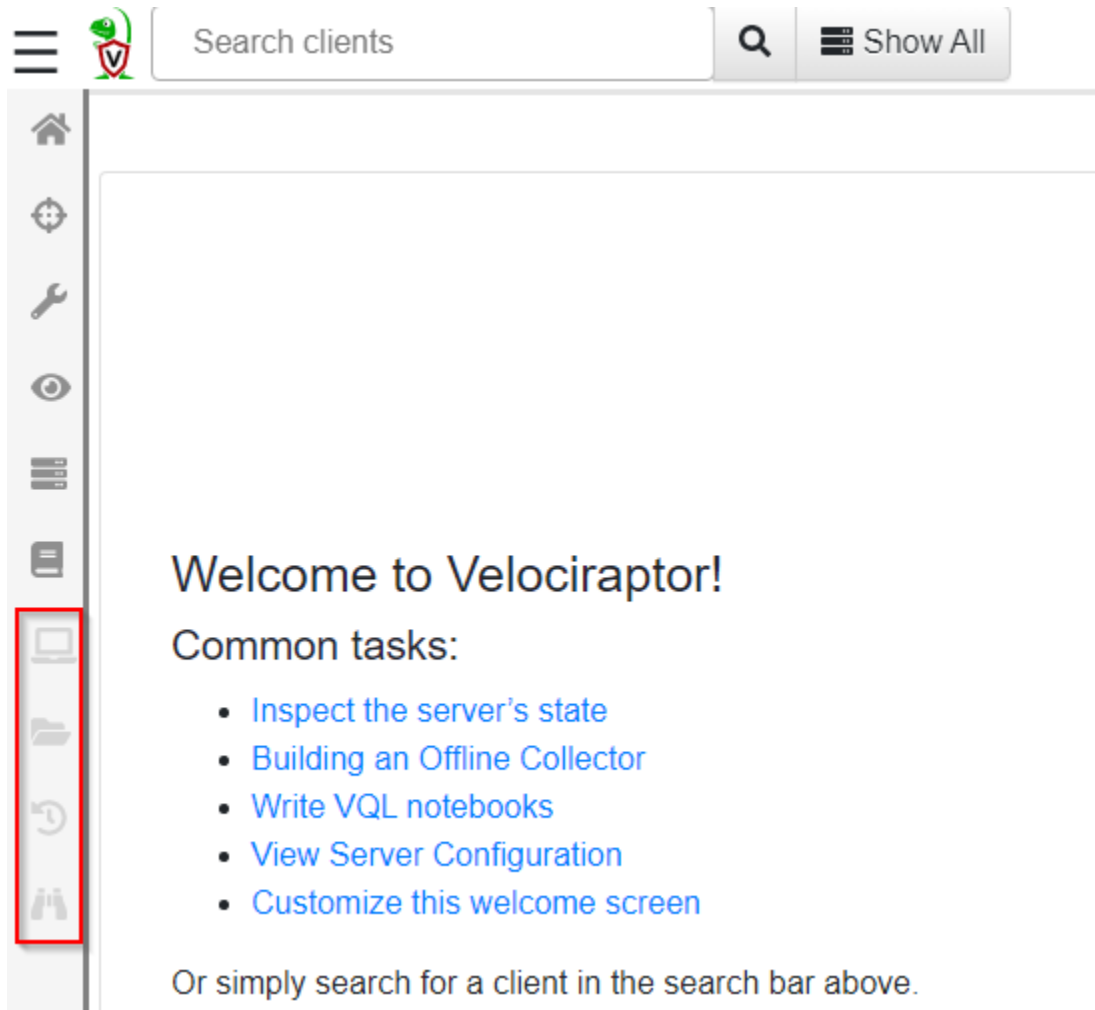
This launches an Instant Velociraptor instance on Windows with the local GUI.

Answer: velociraptor.exe gui

Task 3 – Interacting with client machines

Inspecting Clients

If you didn't notice, some links are grayed out when you first log into Velociraptor. See below.



These links are specific to client endpoints and will become active once the analyst interacts with these endpoints within the Velociraptor UI.

Let's add a client to Velociraptor. Remember, since the attached VM is running Windows Subsystem for Linux (WSL), the Velociraptor server is running in Ubuntu, but the client will be Windows.

Run the commands for 'Add Windows as a client (CMD)' from the commands.txt on the desktop.

Click on the agent to bring you to a semi-detailed view. By default, the view shown is the **overview** for the client.

Overview

In this view, the analyst (you) will see additional information about the client. The additional details are listed below:

- **Client ID**
- **Agent Version**
- **Agent Name**
- **Last Seen At**
- **Last Seen IP**
- **Operating System**
- **Hostname**
- **Release**
- **Architecture**
- **Client Metadata**

VQL Drilldown

In this view, there is additional information about the client, such as Memory and CPU usage over 24 hours timespan, the Active Directory domain if the client is a domain-joined machine and the active local accounts for the client.

The data is represented in two colors in the Memory and CPU footprint over the past 24 hours.

- **Orange** - Memory usage
- **Blue** - CPU usage

Shell

With the shell, commands can be executed remotely on the client machine. Commands can be run in **PowerShell**, **CMD**, **Bash**, or **VQL**. Depending on the target operating system will determine which the analyst will pick. For example, CMD will not be a viable option if the client machine is running Linux.

In the example below, the command `whoami` was executed with PowerShell. The command results are not immediately visible, and the **eyeball** icon needs to be toggled to see the command results.



Across the top pane are brief details of the 'collected' artifact. See below.

Clicking on any FlowId will populate the bottom pan with additional details regarding the information collected for that artifact or collection.

Artifact Collection		Uploaded Files	Requests	Results	Log	Notebook					
Overview											
Artifact Names		Generic.Client.Info									
Flow ID		F.CAD870A0KOIEA									
Creator											
Create Time		2022-06-03 21:56:17 UTC									
Start Time		2022-06-03 21:56:18 UTC									
Last Active		2022-06-03 21:56:18 UTC									
Duration		0.22 Seconds									
State		FINISHED									
Ops/Sec		Unlimited									
Timeout		600 seconds									
Max Rows		1m rows									
Max Mb		1000.00 Mb									
Parameters											
Generic.Client.Info											
Results											
Artifacts with Results		Generic.Client.Info/UsersGeneric.Client.Info/BasicInformation									
Total Rows		8									
Uploaded Bytes		0 / 0									
Files uploaded		0									
Download Results											
Available Downloads											
Name ↕		Size (Mb) ↕			Date						

This section is very busy, and I'll leave you to acquaint yourself with the information displayed here for each collected artifact.

The questions in this task will help nudge you to navigate throughout the output returned for each shell execution (e.i. whoami).

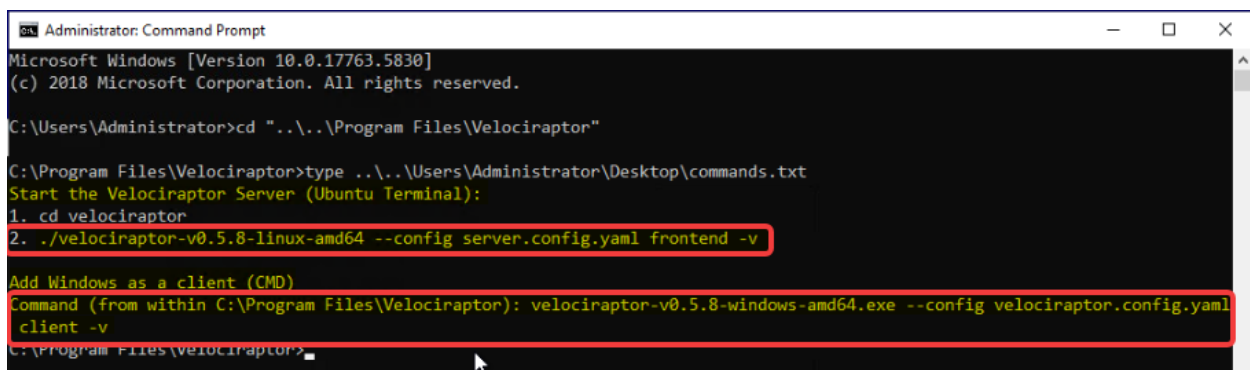
In the next task, we'll explore how to create a new collection and review the results in Collected.

Interrogate

Per the [documentation](#), "Interrogation is the term we use to describe the process of querying a host for it's basic host information".

To confirm this, click Interrogate. Now navigate back to Collected. You will notice that the **Artifact Collection** is **Generic. Client.Info**, which is an additional collection on the list. The first artifact collection in the list is indeed **Generic.Client.Info**. This is the same information displayed under **VQL Drilldown**.

Answer the questions below



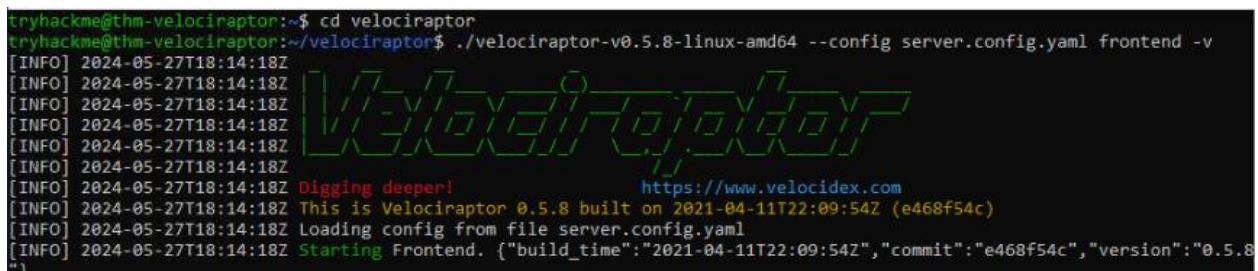
```
Administrator: Command Prompt
Microsoft Windows [Version 10.0.17763.5830]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Users\Administrator>cd "..\..\Program Files\Velociraptor"

C:\Program Files\Velociraptor>type ..\..\Users\Administrator\Desktop\commands.txt
Start the Velociraptor Server (Ubuntu Terminal):
1. cd velociraptor
2. ./velociraptor-v0.5.8-linux-amd64 --config server.config.yaml frontend -v

Add Windows as a client (CMD)
Command (from within C:\Program Files\Velociraptor): velociraptor-v0.5.8-windows-amd64.exe --config velociraptor.config.yaml client -v
C:\Program Files\Velociraptor>
```

I opened the commands.txt file, which contains the commands that will be used later to start the Velociraptor server and register the Windows machine as a client.

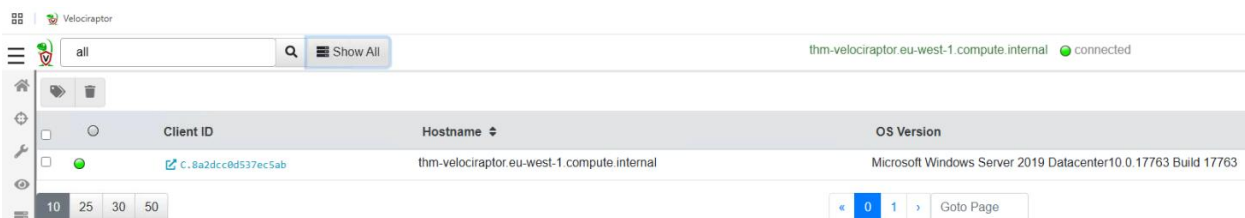


```
tryhackme@thm-velociraptor:~$ cd velociraptor
tryhackme@thm-velociraptor:~/velociraptor$ ./velociraptor-v0.5.8-linux-amd64 --config server.config.yaml frontend -v
[INFO] 2024-05-27T18:14:18Z
[INFO] 2024-05-27T18:14:18Z
[INFO] 2024-05-27T18:14:18Z
[INFO] 2024-05-27T18:14:18Z
[INFO] 2024-05-27T18:14:18Z
[INFO] 2024-05-27T18:14:18Z
[INFO] 2024-05-27T18:14:18Z Digging deeper! https://www.velocidex.com
[INFO] 2024-05-27T18:14:18Z This is Velociraptor 0.5.8 built on 2021-04-11T22:09:54Z (e468f54c)
[INFO] 2024-05-27T18:14:18Z Loading config from file server.config.yaml
[INFO] 2024-05-27T18:14:18Z Starting Frontend. {"build_time":"2021-04-11T22:09:54Z","commit":"e468f54c","version":"0.5.8"}
```

I started the Velociraptor server using the server configuration file. The output confirms that the configuration was loaded successfully and the frontend service started, allowing clients to connect.

```
C:\Program Files\Velociraptor>velociraptor-v0.5.8-windows-amd64.exe --config velociraptor.config.yaml client -v
[INFO] 2024-05-27T18:16:51Z
[INFO] 2024-05-27T18:16:51Z
[INFO] 2024-05-27T18:16:51Z
[INFO] 2024-05-27T18:16:51Z
[INFO] 2024-05-27T18:16:51Z
[INFO] 2024-05-27T18:16:51Z
[INFO] 2024-05-27T18:16:51Z Digging deeper! https://www.velocidex.com
[INFO] 2024-05-27T18:16:51Z This is Velociraptor 0.5.8 built on 2021-04-11T22:11:10Z (e468f54c)
[INFO] 2024-05-27T18:16:51Z Loading config from file velociraptor.config.yaml
[INFO] 2024-05-27T18:16:51Z Loading writeback from C:\Program Files\Velociraptor\velociraptor.writeback.yaml
[INFO] 2024-05-27T18:16:51Z Setting temp directory to C:\Program Files\Velociraptor\Tools
[INFO] 2024-05-27T18:16:51Z Starting Crypto for client C.8a2dcc0d537ec5ab
[INFO] 2024-05-27T18:16:51Z Starting Journal service.
[INFO] 2024-05-27T18:16:51Z Starting the notification service.
[INFO] 2024-05-27T18:16:51Z Installing Dummy inventory service. Will download tools to temp directory.
[INFO] 2024-05-27T18:16:51Z Loaded 250 built in artifacts in 131.6253ms
[INFO] 2024-05-27T18:16:51Z Starting event query service with version 0.
[INFO] 2024-05-27T18:16:51Z Starting event query service with version 1716720681683946300.
[INFO] 2024-05-27T18:16:51Z Expecting self signed certificate for server.
[INFO] 2024-05-27T18:16:51Z Starting monitoring query $f169a393d57b85f86c362e4457367e015d4bd7268a1dde5e8aac4791529a3854
[INFO] 2024-05-27T18:16:51Z vql: Starting query execution.
[INFO] 2024-05-27T18:16:51Z Ring Buffer: Creation {"filename":"C:\\Program Files\\Velociraptor\\Tools\\Velociraptor_Buffer.bin","max_size":1073741874}
```

I started the Velociraptor client using the shared configuration file. The terminal output confirms that the client initialized its services and successfully connected to the Velociraptor server.



The screenshot shows the Velociraptor web interface in a browser. At the top, there's a search bar with 'all' and a 'Show All' button. To the right, it says 'thm-velociraptor.eu-west-1.compute.internal' with a green 'connected' status. Below this is a table with columns: Client ID, Hostname, and OS Version. There is one row with a green status indicator, Client ID 'C.8a2dcc0d537ec5ab', Hostname 'thm-velociraptor.eu-west-1.compute.internal', and OS Version 'Microsoft Windows Server 2019 Datacenter10.0.17763 Build 17763'. At the bottom, there are pagination controls showing '10', '25', '30', '50' and a 'Goto Page' button.

Client ID	Hostname	OS Version
C.8a2dcc0d537ec5ab	thm-velociraptor.eu-west-1.compute.internal	Microsoft Windows Server 2019 Datacenter10.0.17763 Build 17763

I opened the Velociraptor web interface in Chrome after starting the server and client. The dashboard shows the Windows endpoint with a green status indicator, confirming that the client connected successfully to the Velociraptor server.

What is the hostname for the client?

thm-velociraptor.eu-west-1.compute.internal	
Client ID	C.8a2dcc0d537ec5ab
Agent Version	2021-04-11T22:11:10Z
Agent Name	velociraptor
Last Seen At	2024-05-27 19:00:27 UTC
Last Seen IP	[::1]:50395
Operating System	windows
Hostname	thm-velociraptor.eu-west-1.compute.internal
Release	Microsoft Windows Server 2019 Datacenter10.0.17763 Build 17763
Architecture	amd64

I opened the client details in the Velociraptor dashboard to review the endpoint information. The hostname displayed for the connected client is **thm-velociraptor.eu-west-1.compute.internal**.

Answer: THM-VELOCIRAPTOR.eu-west-1.compute.internal

What is listed as the agent version?

thm-velociraptor.eu-west-1.compute.internal	
Client ID	C.8a2dcc0d537ec5ab
Agent Version	2021-04-11T22:11:10Z
Agent Name	velociraptor
Last Seen At	2024-05-27 19:00:27 UTC
Last Seen IP	[::1]:50395
Operating System	windows
Hostname	thm-velociraptor.eu-west-1.compute.internal
Release	Microsoft Windows Server 2019 Datacenter10.0.17763 Build 17763
Architecture	amd64

I reviewed the client details in the Velociraptor dashboard. The endpoint information shows that the installed Velociraptor agent version is **2021-04-11T22:11:10Z**.

Answer: 2021-04-11T22:11:10Z

In the Collected tab, what was the VQL command to query the client user accounts?

State	FlowId	Artifacts	Created	Last Active	Creator
✓	F.CPADNJC7705IA	Generic.Client.Info	2024-05-27 19:15:25 UTC	2024-05-27 19:15:26 UTC	thmadmin
✓	F.CPADLTL8GCHDM	Generic.Client.Info	2024-05-27 19:11:50 UTC	2024-05-27 19:11:51 UTC	thmadmin
✓	F.CPAD2H8O1C7RK	Windows.System.PowerShell	2024-05-27 18:30:29 UTC	2024-05-27 18:30:30 UTC	thmadmin

Artifact CollectionUploaded FilesRequestsResultsLogNotebook

Request sent to client

```
17 },
18 {
19   "query_id": "1",
20   "urgent": true,
21   "task_id": "1716837325268929",
22   "VQLClientAction": {
23     "precondition": "SELECT OS From Info() where OS = 'windows'",
24     "Query": {
25       {
26         "VQL": "LET precondition_Generic_Client_Info_Users_0=SELECT OS FROM Info() WHERE OS = 'windows'"
27       },
28       {
29         "VQL": "LET Generic_Client_Info_Users_0=SELECT Name, Description, Mtime AS LastLogin FROM Artifact.Windows.Sys.Users()"
30       },
31     },
32     "Name": "24b3883c0508bda3c4a73894cf3e1d4c63c719338d36b4ea3cad428fa2cf79",
33     "VQL": "SELECT * FROM IF($pre=Generic_Client_Info_Users_0, condition=precondition_Generic_Client_Info_Users_0, else= { SELECT * FROM scope() WHERE log(message='Query skipped due to precondition') AND FALSE})"
34   },
35   "max_row": "1000",
36   "artifacts": [
37     {
38       "name": "Windows_Sys_Users"
```

I opened the **Collected** tab and selected the **Requests** tab to review the VQL queries sent to the client. The query used to retrieve the client user accounts is shown in the request details.

Answer: LET Generic_Client_Info_Users_0_0=SELECT Name, Description, Mtime AS LastLogin FROM Artifact.Windows.Sys.Users()

In the Collected tab, check the results for the PowerShell whoami command you executed previously. What is the column header that shows the output of the command?

State	FlowId	Artifacts	Created	Last Active	Creator	Mb	Rows
✓	F.CPADNJC7705IA	Generic.Client.Info	2024-05-27 19:15:25 UTC	2024-05-27 19:15:26 UTC	thmadmin		8
✓	F.CPADLTL8GCHDM	Generic.Client.Info	2024-05-27 19:11:50 UTC	2024-05-27 19:11:51 UTC	thmadmin		8
✓	F.CPAD2H8O1C7RK	Windows.System.PowerShell	2024-05-27 18:30:29 UTC	2024-05-27 18:30:30 UTC	thmadmin		1

Artifact CollectionUploaded FilesRequestsResultsLogNotebook

Windows System PowerShell

Stdout	Stderr	ReturnCode	Complete
nt authority\system		0	true

10253050

Showing rows 1 to 1 of 1

0

Goto Page

Answer: Stdout

In the Shell, run the following PowerShell command Get-Date. What was the PowerShell command executed with VQL to retrieve the result?

State	FlowId	Artifacts	Created	Last Active	Creator
✓	F.CPAE9B87RQEU8	Windows.System.PowerShell	2024-05-27 19:53:17 UTC	2024-05-27 19:53:18 UTC	thmadmin
✓	F.CPADNJC7705IA	Generic.Client.Info	2024-05-27 19:15:25 UTC	2024-05-27 19:15:26 UTC	thmadmin
✓	F.CPADLTL8GCHDM	Generic.Client.Info	2024-05-27 19:11:50 UTC	2024-05-27 19:11:51 UTC	thmadmin

Artifact Collection

Uploaded Files

Requests

Results

Log

Notebook

client_time	message
2024-05-27 19:53:17 UTC	vql: Starting query execution.
2024-05-27 19:53:17 UTC	vql: shell: Running external command [powershell -ExecutionPolicy Unrestricted -encodedCommand RwbIAHQALQBEAGEAdABIAA==]
2024-05-27 19:53:18 UTC	Time 0: Windows.System.PowerShell: Sending response part 0 827 B (1 rows).
2024-05-27 19:53:18 UTC	vql: Collection is done after 879.3314ms
2024-05-27 19:53:18 UTC	vql: Query Stats: {"RowsScanned":4,"PluginsCalled":4,"FunctionsCalled":2,"ProtocolSearch":0,"ScopeCopy":15}

I ran the Get-Date command from the Velociraptor Shell and then opened the **Log** tab for the **Windows.System.PowerShell** artifact. The log shows that VQL executed PowerShell using the -ExecutionPolicy Unrestricted and -encodedCommand parameters, with the command passed as a Base64-encoded string.

Answer: powershell -ExecutionPolicy Unrestricted -encodedCommand
ZwBIAHQALQBkAGEAdABIAA==

Task 4 – Creating a new collecion

Creating a new collection

In this task let's create a new collection.



We will take advantage of the WSL set-up in the attached VM and choose an artifact specific to Ubuntu.

There will be 5 stages in this process.

- **Select Artifacts**
- **Configure Parameters**
- **Specify Resources**
- **Review**
- **Launch**

Select Artifacts

From the left-hand menu, navigate to **Collected Artifacts** (not **Server Artifacts**). Click the “+” button, then type `Windows.KapeFiles.Targets` in the search bar. If you're not familiar with KAPE, please visit the [KAPE room](#).

In short, **KapeFiles** are community-created targets and modules for use with KAPE. But as you can see, other tools use these Kapefiles as well.

When you select the artifact, a brief description of the collector will be displayed on the right, along with a rundown of the parameters.

New Collection: Select Artifacts to collect

windows.k Windows.Forensics.Prefetch Windows.KapeFiles.Targets Windows.Timeline.Prefetch	Windows.KapeFiles.Targets Type: client Kape is a popular bulk collector tool for triaging a system quickly. While KAPE itself is not an opensource tool, the logic it uses to decide which files to collect is encoded in YAML files hosted on the KapeFiles project (https://github.com/EricZimmerman/KapeFiles) and released under an MIT license. This artifact is automatically generated from these YAML files, contributed and maintained by the community. This artifact only encapsulates the KAPE "Targets" - basically a bunch of glob expressions used for collecting files on the endpoint. We do not do any post processing these files - we just collect them. We recommend that timeouts and upload limits be used conservatively with this artifact because we can upload really vast quantities of data very quickly.
---	---

Configure Parameters

New Collection: Configure Parameters

-	Artifact
-	Windows.KapeFiles.Targets
UseAutoAccessor	☒ Uses file accessor when possible instead of ntfs parser - this is much faster.
Device	C:
VSSAnalysis	☐ If set we run the collection across all VSS and collect only unique changes.

Scroll down and check **Ubuntu**.

Ubuntu ☐ Ubuntu on Windows Subsystem for Linux (by Matt Dawson): Ubuntu WSL .bashrc, Ubuntu WSL .profile, Ubuntu WSL User Crontabs, Ubuntu WSL Apt Logs, Ubuntu WSL /etc/os-release, Ubuntu WSL /etc/fstab, Ubuntu WSL /etc/passwd, Ubuntu WSL /etc/group, Ubuntu WSL /etc/shadow, Ubuntu WSL /etc/timezone, Ubuntu WSL /etc/hostname, Ubuntu WSL /etc/hosts, Ubuntu WSL /etc/crontab, Ubuntu WSL /etc/bash.bashrc, Ubuntu WSL /etc/profile, Ubuntu WSL .bash_history

Next, click on **Specify Resources**.

Specify Resources

You can leave this untouched. See the below screenshot.

New Collection: Specify Resources

Ops/Sec

Unlimited

Max Execution Time in Seconds

600s per artifact

Max Rows

1,000,000 rows

Max Mb Uploaded

1Gb

Next, click on **Review**.

Review

The output will display in JSON format and it's pretty straightforward. Only one setting was enabled to collect, which was Ubuntu.

New Collection: Review request

```
1 {
2   "artifacts": [
3     "Windows.KapeFiles.Targets"
4   ],
5   "specs": [
6     {
7       "artifact": "Windows.KapeFiles.Targets",
8       "parameters": {
9         "env": [
10          {
11            "key": "Ubuntu",
12            "value": "Y"
13          }
14        ]
15      }
16    }
17  ]
18 }
```

Launch

Everything should be in order. Now it's time to launch the collection to gather the artifacts.

When you click **Launch**, you will be redirected to the Collected view. Notice that there should be a new entry with the newly created collection.

In particular, notice the State. It should show an hourglass which indicates the artifacts are actively being gathered for that collection.

State ▾	FlowId	Artifacts ▾
	F.C95MTTEA835F4	Windows.KapeFiles.Targets

Once the artifacts have been gathered, the state will change from an hourglass to a checkmark like the others.

State ▾	FlowId	KAPEFILES ▾
	F.C95MTTEA835F4	Windows.KapeFiles.Targets

As the list of collections grows, you can search for specific collections using the textfield at the top of the column. See the above screenshot.

Sweet! Now that we got that covered, let's look at VFS.

Refer to the Velociraptor documentation to learn more about [Artifacts \(opens in new tab\)](#).

Answer the questions below

Earlier you created a new artifact collection for Windows.KapeFiles.Targets. You configured the parameters to include Ubuntu artifacts. Review the parameter description for this setting. What is this parameter specifically looking for?

Ubuntu ☒ **Ubuntu on Windows Subsystem for Linux** (by Matt Dawson): Ubuntu WSL .bashrc, Ubuntu WSL .profile, Ubuntu WSL User Crontabs, Ubuntu WSL Apt Logs, Ubuntu WSL /etc/os-release, Ubuntu WSL /etc/fstab, Ubuntu WSL /etc/passwd, Ubuntu WSL /etc/group, Ubuntu WSL /etc/shadow, Ubuntu WSL /etc/timezone, Ubuntu WSL /etc/hostname, Ubuntu WSL /etc/hosts, Ubuntu WSL /etc/crontab, Ubuntu WSL /etc/bash.bashrc, Ubuntu WSL /etc/profile, Ubuntu WSL .bash_history

I reviewed the description of the **Ubuntu** parameter under the **Windows.KapeFiles.Targets** artifact. It is specifically looking for Ubuntu installations running through **Windows Subsystem for Linux (WSL)**.

Answer: Ubuntu on Windows Subsystem for Linux

Review the output. How many files were uploaded?

The screenshot shows the Velociraptor web interface. At the top, there's a search bar and a status bar indicating 'thm-velociraptor.eu-central-1.compute.internal' is connected. Below this is a table of artifact collections. The first row is highlighted in yellow and shows 'F.CPAGVCDIA8SCQ' for 'Windows.KapeFiles.Targets', created on '2024-05-27 22:36:49 UTC', last active on '2024-05-27 22:36:53 UTC', created by 'thmadmin', with 0 MB and 38 rows. Below the table, there are tabs for 'Artifact Collection', 'Uploaded Files', 'Requests', 'Results', 'Log', and 'Notebook'. The 'Results' tab is selected, showing a summary of the artifact collection. It lists 'Artifact Names' as 'Windows.KapeFiles.Targets', 'Flow ID' as 'F.CPAGVCDIA8SCQ', 'Creator' as 'thmadmin', 'Create Time' as '2024-05-27 22:36:49 UTC', 'Start Time' as '2024-05-27 22:36:50 UTC', 'Last Active' as '2024-05-27 22:36:53 UTC', 'Duration' as '3.56 Seconds', 'State' as 'FINISHED', and 'Ops/Sec' as 'Unlimited'. On the right, under 'Results', it shows 'Artifacts with Results' as 'Windows.KapeFiles.Targets/All File Metadata/Windows.KapeFiles.Targets/Uploads', 'Total Rows' as 38, 'Uploaded Bytes' as 20951 / 20951, and 'Files uploaded' as 19, which is highlighted in yellow. Below this, there's a section for 'Available Downloads' with columns for 'Name', 'Size (Mb)', and 'Date'.

I reviewed the completed **Windows.KapeFiles.Targets** artifact collection. The overview shows that **19 files** were successfully uploaded during the collection process.

Answer: 19

Task 5 – VFS (Virtual File System)

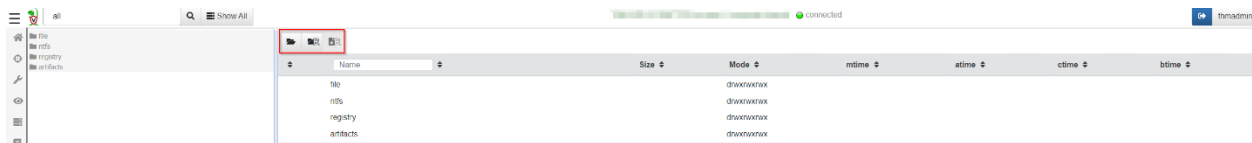
The Virtual File System

Per the [documentation](#), "Velociraptor provides an interface to interact with the client's filesystem which we call the Virtual File System or VFS. The VFS GUI is a convenient tool to interactively inspect the client's filesystem and fetch files if necessary."

This can prove useful in an incident response scenario where you, the analyst, need to inspect artifacts in a client.

Refer to the official documentation for a complete overview of the VFS. In this task, we're going to focus on getting hands-on with VFS.

Below is what you should see when you first access the VFS for a client.



In the left pane, along with the middle pane, there are 4 folders (or accessors, filesystem access drivers):

- **file** - uses operating system APIs to access files
- **ntfs** - uses raw NTFS parsing to access low level files
- **registry** - uses operating system APIs to access the Windows registry
- **artifacts** - previously run collections.

Three buttons are highlighted in the above image. Below is a brief explanation for each.



1. Refresh the current directory (sync its listing from the client)
2. Recursively refresh this directory (sync its listing from the client)
3. Recursively download this directory from the client

Let's continue interacting with VFS.

When any folder is clicked in the left pane, additional details are displayed in the middle pane. For example, if the file folder is clicked, a subfolder will appear, which is **C:.** Now the details in the middle pane change to reflect C:.

Answer the questions below

Which accessor can access hidden NTFS files and Alternate Data Streams? (**format: xyz accessor**)

<https://docs.velociraptor.app/docs/clients/vfs/>

The **ntfs accessor** can access hidden NTFS files and Alternate Data Streams directly from the filesystem.

Answer: ntfs accessor

Which accessor provides file-like access to the registry? (**format: xyz accessor**)

<https://docs.velociraptor.app/docs/clients/vfs/>

The **registry accessor** provides file-like access to the Windows Registry.

Answer: registry accessor

What is the name of the file in \$Recycle.Bin?



I browsed to the user SID folder inside **\$Recycle.Bin** using the file accessor. The file listed in the directory is **desktop.ini**.

Answer: desktop.ini

Answer: THM{VkVMT0NJuKfQVE9S}

Velociraptor Query Language

With many tools that you will encounter in your SOC career, some tools may have their own query language. For example, in Splunk its SPL ([Search Processing Language](#)), Elastic has KQL ([Query Language](#)), Microsoft Sentinel has KQL [too] ([Kusto Query Language](#)), etc.

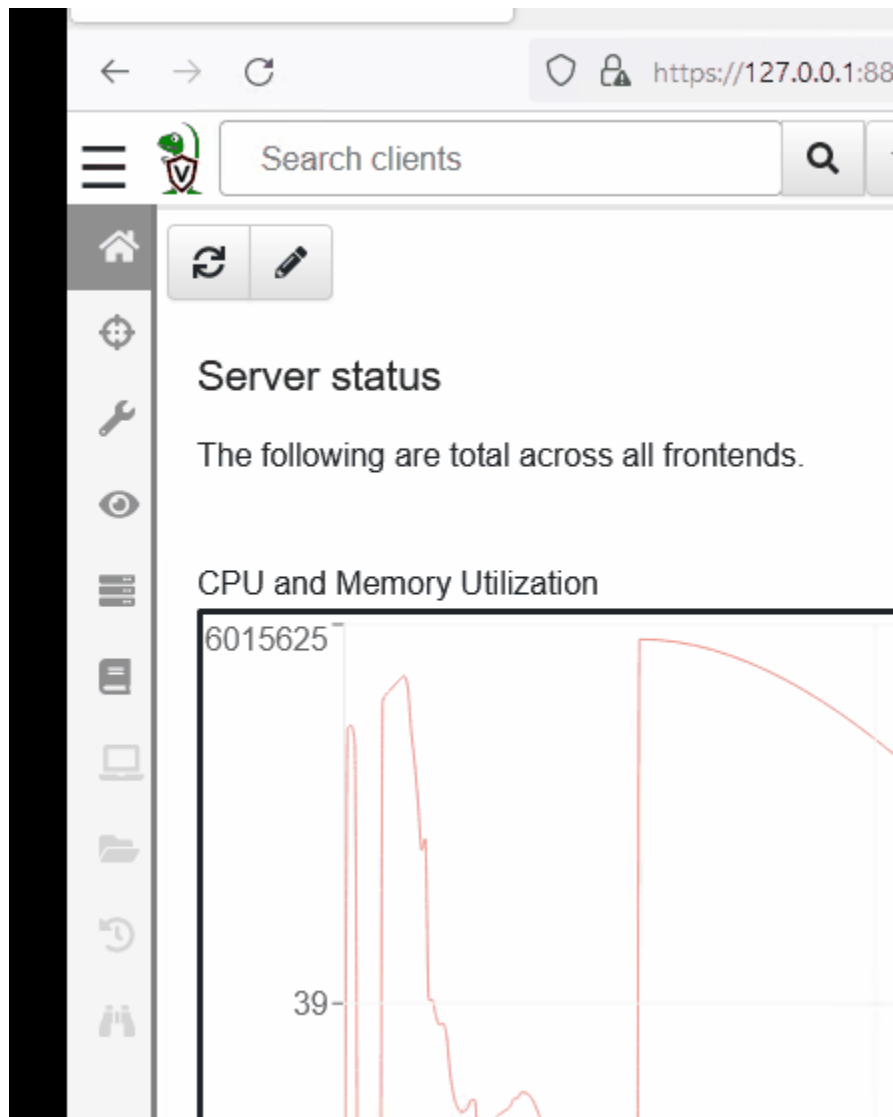
To jog your memory, navigate back to **Collected** and inspect **Generic.Client.Info**. Click the Requests tab in the bottom pane. See below image.

```
"VOL": "LET Generic_Client_Info_Users_0 0=SELECT Name, Description, Mtime AS LastLogin FROM Artifact.Windows.Sys.Users()"
```

If you are familiar with SQL (Structured Query Language) then you should notice the similarities, for example: **SELECT**, **FROM**, and **WHERE**.

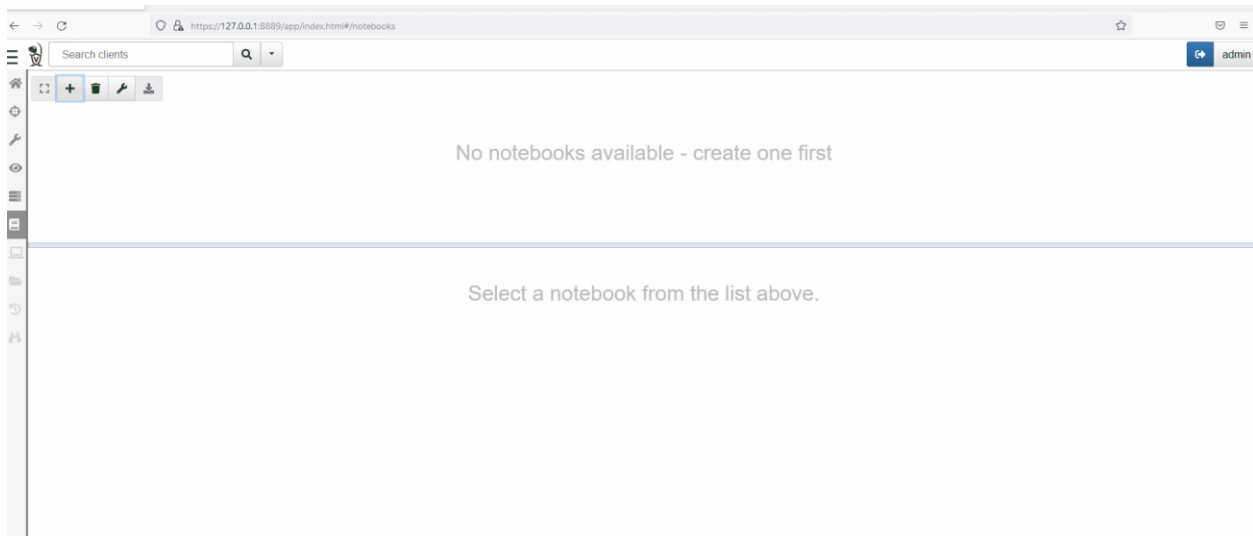
To execute a simple VQL on your own, first create a [Notebook \(opens in new tab\)](#).

Navigate to the **Notebooks** tab. In Velociraptor, **Notebooks** are *containers* that we can use to execute our queries and commands, as demonstrated below.



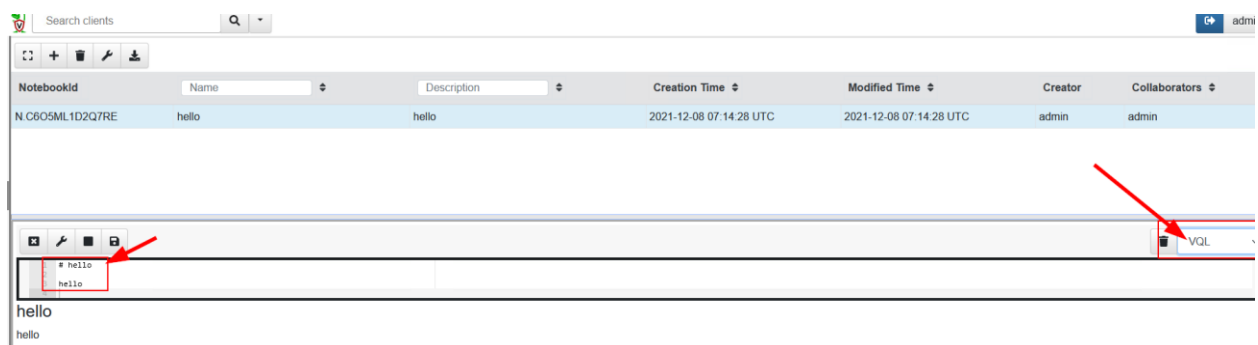
Notebooks consist of two languages - [Markdown \(opens in new tab\)](#) and (of course) **VQL**. If you are familiar with [Jupyter Notebooks \(opens in new tab\)](#) they function in a very similar fashion!

Let's create our first notebook and enter some simple markdown. We'll circle back to VQL shortly.

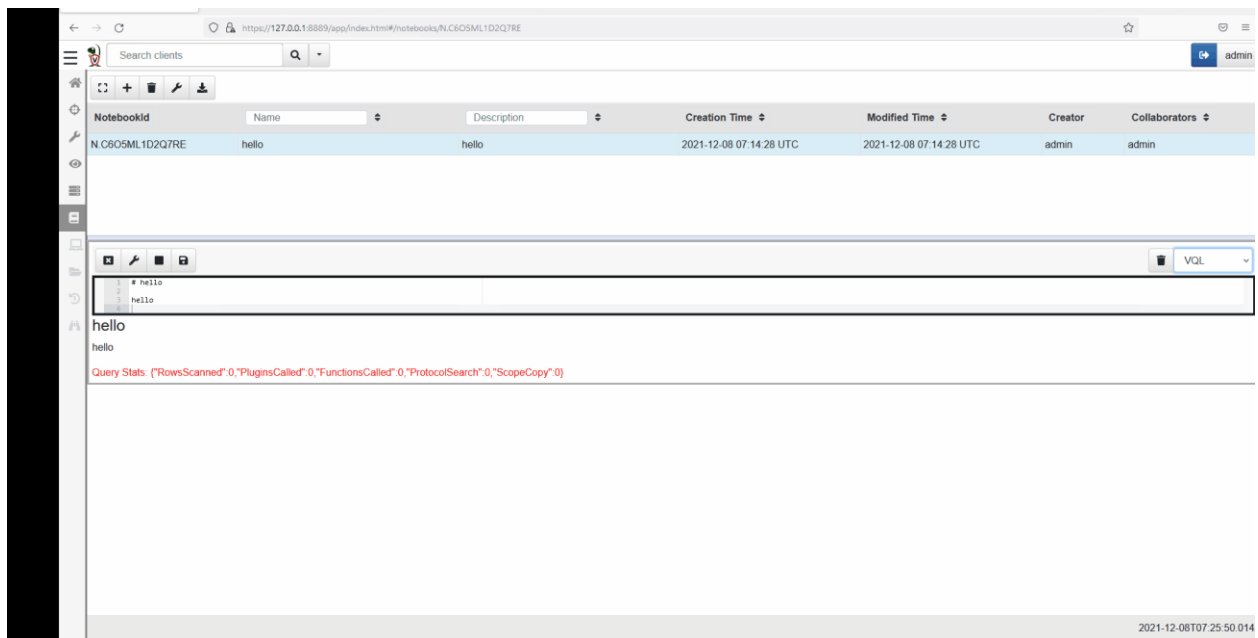


Sweet! Now let's set our notebook to use VQL instead. To query basic information from the current agent, we can use `SELECT * FROM info()`

Note: Click into the lower box to display the options for this, then select the pencil to edit.

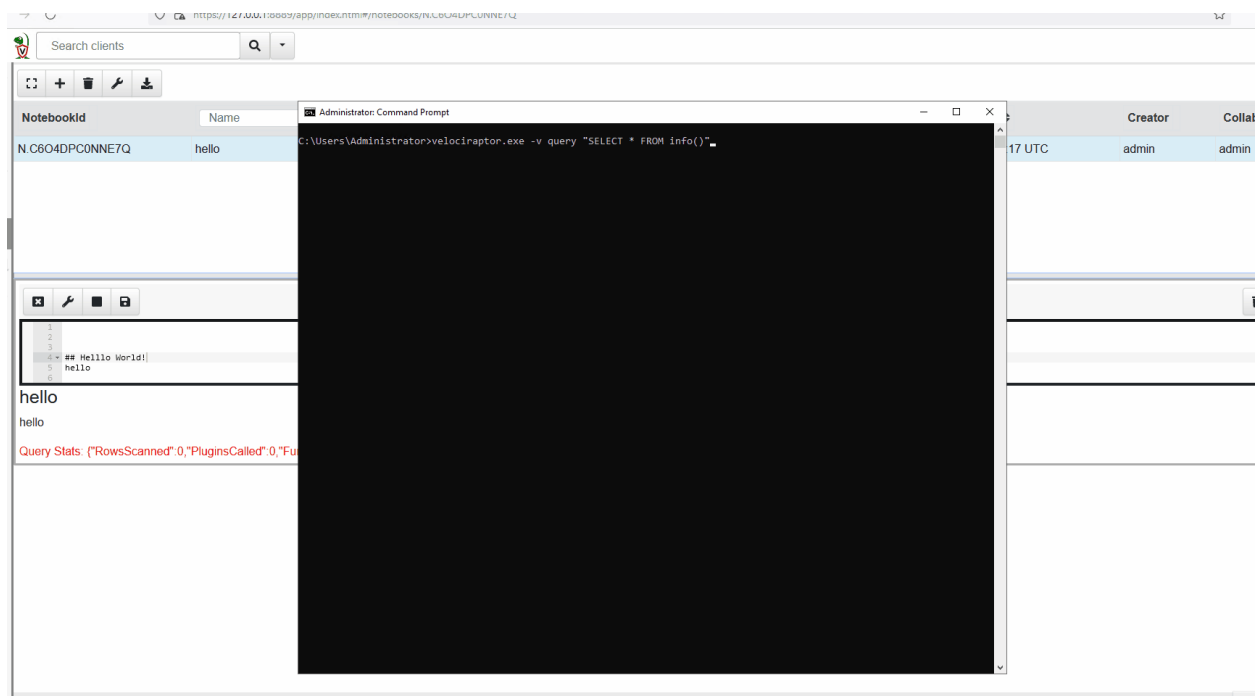


Let's save this notebook and run it against the agent as demonstrated below.



VQL can also be run via the command line. See the example below.

For this example, VQL is run from the command line querying an agent for details such as its hostname.



Artifacts

Before wrapping up this task, let's touch on **Artifacts** (or VQL Modules).

Per the [documentation](#), "Velociraptor allows packaging VQL queries inside mini-programs called Artifacts. An artifact is simply a structured file containing a query, with a name attached to it. This allows Velociraptor users to search for the query by name or description and simply run the query on the endpoint without necessarily needing to understand or type the query into the UI".

This was a **BRIEF** intro to VQL. It is recommended to review the official [documentation](#) thoroughly to fully understand it and how you can wield its power to execute advanced queries. Also, reference the [VQL Reference](#) and [Extending VQL](#) for further information on VQL.

Answer the questions below

What is followed after the **SELECT** keyword in a standard VQL query?

<https://docs.velociraptor.app/docs/vql/fundamentals/>

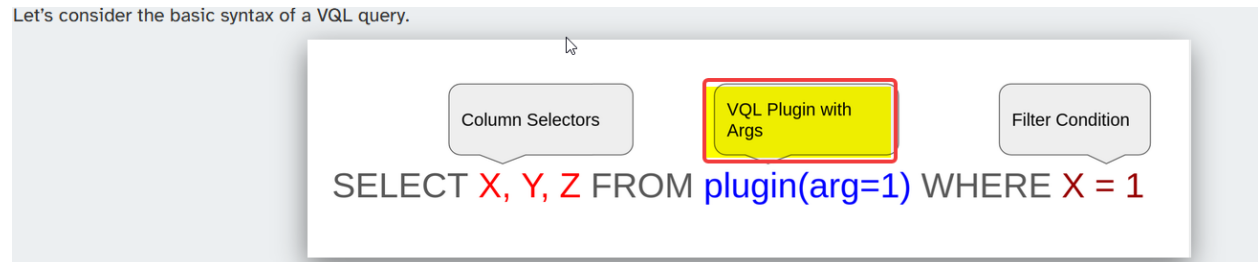
Let's consider the basic syntax of a VQL query.



Answer: Column Selectors

What goes after the **FROM** keyword?

Let's consider the basic syntax of a VQL query.



Answer: VQL Plugin

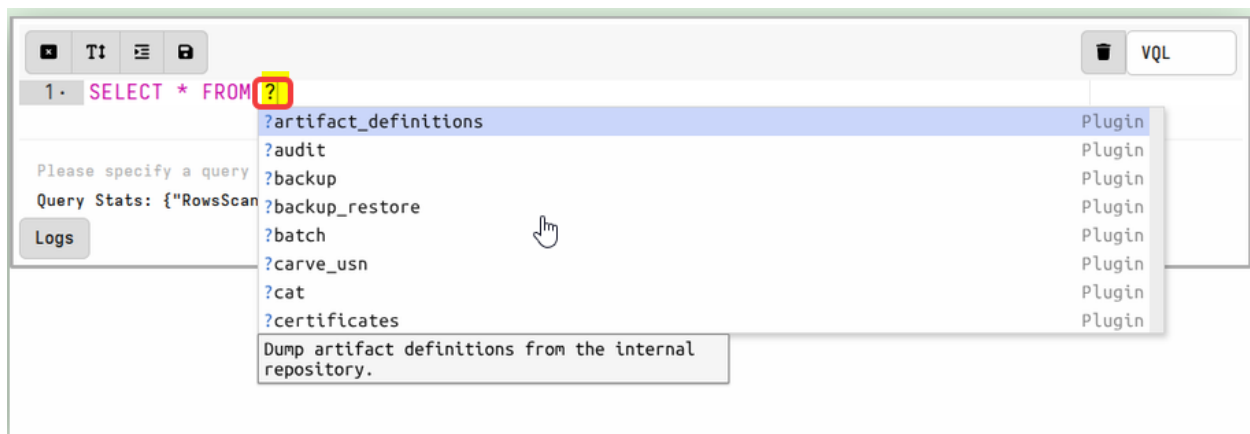
What is followed by the **WHERE** keyword?

Let's consider the basic syntax of a VQL query.



Answer: Filter expression

What can you type in the Notepad interface to view a list of possible completions for a keyword?



Answer: ?

What plugin would you use to run PowerShell code from Velociraptor?

https://docs.velociraptor.app/docs/vql/extending_vql/

Answer: `execve()`

Task 7 – Forensic Analysis VQL Plugins

Forensic Analysis

Per the [documentation \(opens in new tab\)](#), "VQL is not useful without a good set of plugins that make DFIR work possible. Velociraptor's strength lies in the wide array of VQL plugins and functions that are geared towards making DFIR investigations and detections effective".

There is a lot of information to cover here regarding VQL plugins. This task aims to give you enough information regarding these plugins so you can construct your VQL query to hunt for artifacts of a popular exploit known as Printnightmare.

At the date of the entry of this content, below are the categories surrounding forensic analysis:

- **Searching Filenames**
- **Searching Content**
- **NTFS Analysis**
- **Binary Parsing**
- **Evidence of Execution**
- **Event Logs**
- **Volatile Machine State**

Have a skim through **Searching Filenames** and **NTFS Analysis** to provide a solid brain dump to prep you for the questions below and for the next task.

Answer the questions below

As per [Velociraptor's documentation \(opens in new tab\)](#) on NTFS analysis,

What plugin would be used for parsing the Master File Table (MFT)?

Answer: parse_mft

What filter expression will ensure that no directories are returned in the results?

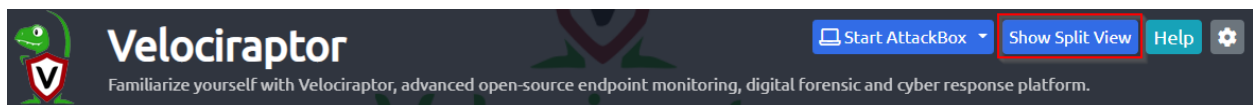
I used a **WHERE** filter expression to exclude directories from the query results. The expression **NOT IsDir** ensures that only files are returned.

Answer: IsDir

Task 8 – Hunt for a nightmare

Hunt for a nightmare

Terminate the previous lab machine and spawn the lab machine attached to this task.



Objective: Use Velociraptor to create an artifact to detect the [PrintNightmare](#) vulnerability!

Luckily there is an artifact entry in the [Artifact Exchange \(opens in new tab\)](#). To avoid just copy/pasting the artifact, you will need to construct a very simple VQL query.

Below are steps to construct your VQL query to find the DLL:

- The **Select** clause, the column accessors should be **fullpath** (concatenate **C:/** to the **fullpath** column accessor) and **filename**.

- Make sure the column headers for each column accessor are renamed. **Fullpath** should be **Full_Path**, and for **filename** it should be **File_Name**.
- Use **parse_pe()** to ensure only PE files are returned. (Check the [VQL Reference](#))
- Make sure the column header for this plugin should be renamed to PE.
- The **From** clause should use **parse_mft()**.
- The **Where** clause should not return any directories, only return binaries (PE files) and search the directory where this malicious DLL will most likely be found.

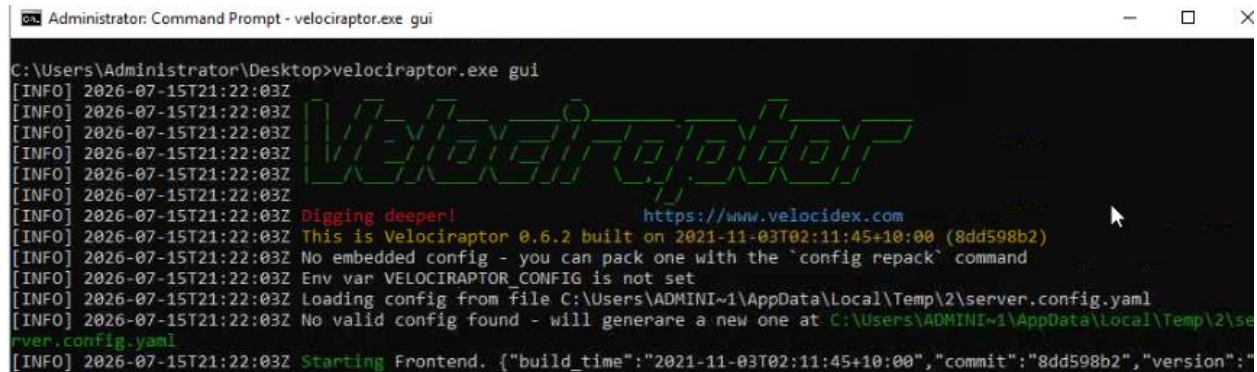
The skeleton query below should help you answer the questions below.

Skeleton Query:

- SELECT "C:/" + FullPath AS *****,FileName AS *****,parse_pe(file="C:/" + FullPath) AS **
- FROM parse_mft(filename="C:/\$***", accessor="****")
- WHERE *** IsDir
- AND FullPath =~ "Windows/System32/spool/drivers"
- AND **

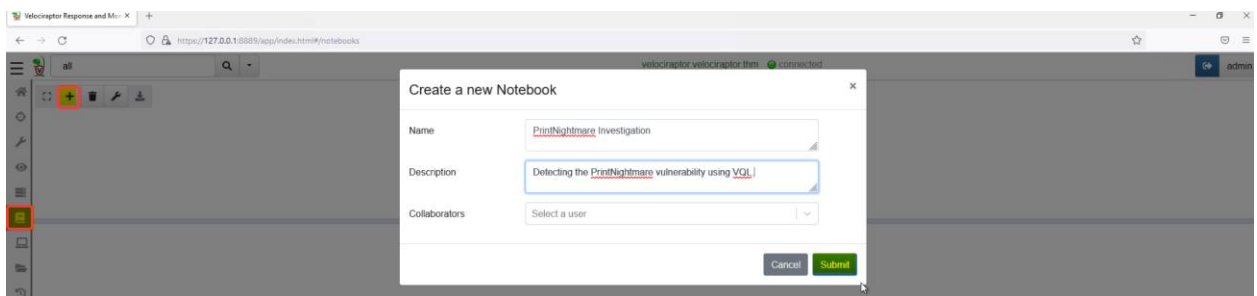
Note: You will need to start Velociraptor in "*Instant Velociraptor*" mode. The instructions to do so can be found [here](#). The lab machine attached to this task is running Velociraptor version **0.6.2**.

Starting Instant Velociraptor



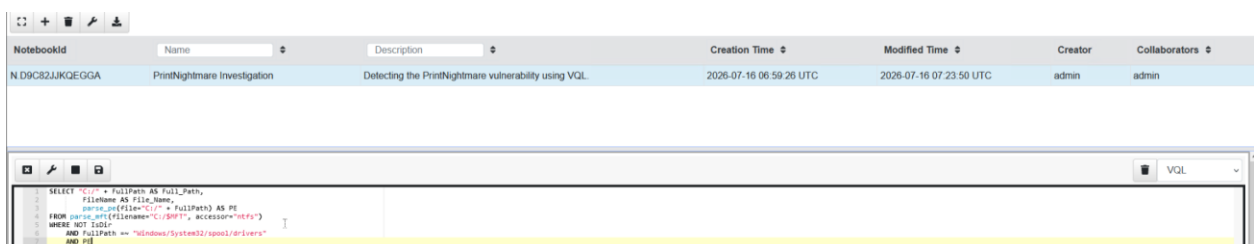
I launched Velociraptor in Instant mode using the `velociraptor.exe gui` command. The output confirms that a temporary configuration file was created and the frontend service started successfully.

Creating a VQL Notebook



I created a new VQL notebook named PrintNightmare Investigation to write and execute the VQL query used during the investigation.

Writing the VQL Query

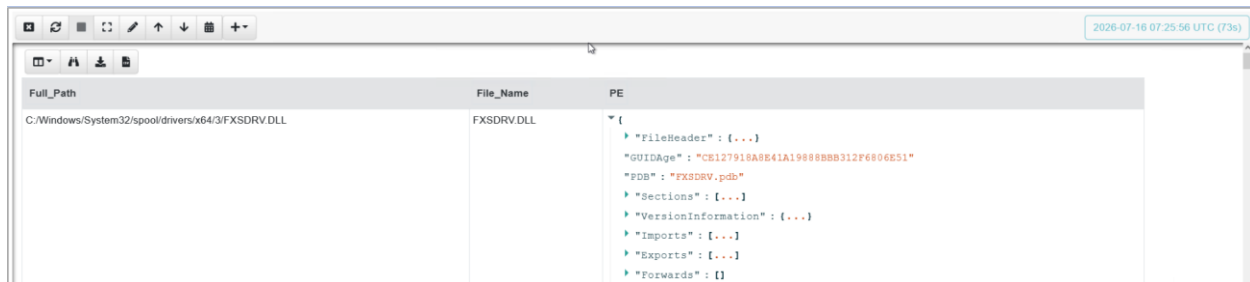


I changed the notebook cell to **VQL** and created a query to search the Windows spool driver directory for PE files by parsing the NTFS Master File Table (MFT).

Query used:

```
SELECT "C:" + FullPath AS Full_Path,  
       FileName AS File_Name,  
       parse_pe(file="C:" + FullPath) AS PE  
FROM parse_mft(filename="C:/$MFT", accessor="ntfs")  
WHERE NOT IsDir  
AND FullPath =~ "Windows/System32/spool/drivers"  
AND PE
```

Running the VQL Query



Full_Path	File_Name	PE
C:/Windows/System32/spool/drivers/x64/3/FXSDRV.DLL	FXSDRV.DLL	<pre>{ "FileHeader": {...} "GUIDAge": "CE127918A8E41A19888BB312F6806E51" "PDB": "FXSDRV.pdb" "Sections": {...} "VersionInformation": {...} "Imports": {...} "Exports": {...} "Forwards": {} }</pre>

I executed the VQL query and reviewed the results. The output lists the matching PE file along with its full path, file name, and parsed PE metadata for further analysis.

Answer the questions below

What is the name in the Artifact Exchange to detect Printnightmare?

I searched the local Velociraptor artifact repository for PrintNightmare but found that the detection artifact was not available in the lab instance. The PrintNightmare detection artifact in the Velociraptor Artifact Exchange is **Windows.Detection.PrintNightmare**.

Answer: Windows.Detection.PrintNightmare

Per the above instructions, what is your Select clause? (no spaces after commas)

Answer: SELECT "C:/\" + FullPath AS Full_Path,FileName AS File_Name,parse_pe(file="C:/\" + FullPath) AS PE

What is the name of the DLL that was placed by the attacker?

The screenshot shows the Velociraptor interface. At the top, a table lists search results. The first row is highlighted:

NotebookId	Name	Description	Creation Time	Modified Time	Creator	Collaborators
N.D9C82JJKQEGGA	PrintNightmare Investigation	Detecting the PrintNightmare vulnerability using VQL	2026-07-16 06:59:26 UTC	2026-07-16 07:23:50 UTC	admin	admin

Below the table, a detailed view of a file's PE metadata is shown for 'nightmare.dll' located at 'C:/Windows/System32/spool/drivers/x64/3/nightmare.dll'. The metadata includes:

- "Exports": {...}
- "Forwards": {}
- "ImpHash": "c0920293bf8b4b7828dbd956a8db729f"
- "Authenticode": null
- "AuthenticodeHash": {...}
- "FileHeader": {...}
- "GUIDAge": "36908f12494b43b9804082d77f38ccb64"
- "PDB": "C:/Users/caleb/source/repos/nightmare/x64/Release/nightmare.pdb"
- "Sections": {...}
- "VersionInformation": {}

The timestamp at the bottom right of the interface is 2026-07-16T07:58:45.486Z.

I reviewed the VQL results and examined the PE metadata for the files in the spool driver directory. Among the listed DLLs, **nightmare.dll** stood out as the suspicious file because its embedded PDB path referenced a developer's build directory, indicating it was likely placed by the attacker.

Answer: nightmare.dll

What is the PDB entry?



I examined the PE metadata for **nightmare.dll** and reviewed its PDB entry. The PDB path points to **C:\Users\caleb\source\repos\nightmare\x64\Release\nightmare.pdb**, indicating the directory where the DLL was built during development.

Answer: C:\Users\caleb\source\repos\nightmare\x64\Release\nightmare.pdb

What is a PDB?

A **PDB (Program Database)** file stores **debugging information** for a compiled Windows program. It helps developers debug applications by including information such as function names, source file references, and symbols.

In DFIR and malware analysis, the PDB path can be very useful because it may reveal:

- The developer's username.
- The project name.
- The build directory.
- Clues about how or where the malware was created.

In this case, the PDB path references a project named **nightmare**, which is another indicator that the DLL is related to the PrintNightmare attack.

Additional Resources:

- <https://docs.velociraptor.app/training/> (opens in new tab)
- <https://docs.velociraptor.app/blog/> (opens in new tab)
- <https://docs.velociraptor.app/presentations/>

Disclaimer

This writeup was created for educational and portfolio purposes only. All testing was performed in the relevant lab environment.

© 2026 Osman Dhaqane. All rights reserved.